



**YOUR COURSE CODE: YOUR COURSE
NAME**

YOUR ASSIGNMENT TITLE

Student Name : YOUR NAME
Matric Number : YOUR MATRIC NUMBER
Course Name : YOUR COURSE NAME (YOUR COURSE CODE)
Course Lecturer : YOUR LECTURER
Last Updated : June 26, 2026

Contents

1	Changing Metadata	1
2	Boxed Environments	2
2.1	Problem Environment	2
2.2	Solution Environment	2
2.3	Other Boxed Environments	2
3	Including MATLAB or Python Code	3
4	Drawing, Figures, and Tables	5
4.1	Plotting Graphs with PGFPlots	5
4.2	Drawing Circuits with CircuiTikZ	5
4.3	Inserting Pre-existing Images	5
4.4	Creating Elegant Tables	6

1 Changing Metadata

To change the assignment details such as the course code, course name, assignment title, and your personal details, simply open `main.tex` and locate the **USER METADATA CONFIGURATION** section at the top.

Modify the following commands with your own details:

- `\newcommand{\mycoursecode}{YOUR COURSE CODE}`
- `\newcommand{\mycoursename}{YOUR COURSE NAME}`
- `\newcommand{\myassignmenttitle}{YOUR ASSIGNMENT TITLE}`
- `\newcommand{\mystudentname}{YOUR NAME}`
- `\newcommand{\mymatricno}{YOUR MATRIC NUMBER}`
- `\newcommand{\mycourselecturer}{YOUR LECTURER}`

These variables will automatically populate the cover page, headers and footers, and the PDF metadata.

2 Boxed Environments

This template includes a beautiful boxed environment for problems and a cleanly separated environment for solutions. It also includes environments for definitions, theorems, and notes.

2.1 Problem Environment

Use the `problem` environment to state your questions. You can provide an optional title to the problem.

Problem 1: Signal Analysis

Calculate the energy and power of the given signal $x(t) = e^{-2t}u(t)$.

2.2 Solution Environment

Following the problem, you can use the `solution` environment to provide your answer. It will automatically style the section and add a QED marker at the end.

Solution:

The energy of the signal is given by:

$$\begin{aligned} E &= \int_{-\infty}^{\infty} |x(t)|^2 dt \\ &= \int_0^{\infty} e^{-4t} dt \\ &= \left[\frac{e^{-4t}}{-4} \right]_0^{\infty} \\ &= \frac{1}{4} \text{ Joules} \end{aligned}$$

Since the energy is finite ($E < \infty$), the power is $P = 0$.



2.3 Other Boxed Environments

You can use the `definition`, `theorem`, and `note` environments to highlight important concepts.

Definition Energy Signal

A signal is an energy signal if its total energy satisfies $0 < E < \infty$.

Theorem Parseval's Theorem

The total energy computed in the time domain is equal to the total energy computed in the frequency domain.

Note Important

Power signals have infinite energy, while energy signals have zero average power.

3 Including MATLAB or Python Code

This template includes a pre-configured `matlabstyle` for `lstlisting` that provides syntax highlighting and a nice background box for your code.

To include code directly in your document, use the `lstlisting` environment and set the style to `matlabstyle`.

```
1 % Define the time vector
2 t = 0:0.01:2*pi;
3
4 % Generate the signal
5 x = sin(2*pi*1*t);
6
7 % Plot the signal
8 figure;
9 plot(t, x, 'LineWidth', 1.5);
10 title('Sine Wave');
11 xlabel('Time (s)');
12 ylabel('Amplitude');
13 grid on;
```

Listing 1. A simple MATLAB script to plot a sine wave.

Even though there's a `matlabstyle`, you can also use the newly added `pythonstyle` to display Python code cleanly with appropriate syntax highlighting:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Define the time vector
5 t = np.linspace(0, 2*np.pi, 100)
6 x = np.sin(2 * np.pi * 1 * t)
7
8 plt.plot(t, x)
9 plt.title('Sine Wave')
10 plt.show()
```

Listing 2. A simple Python script.

We also provide `cppstyle` for C/C++ developers:

```
1 #include <iostream>
2 #include <vector>
3
4 int main() {
5     std::vector<int> numbers = {1, 2, 3, 4, 5};
6     for (int num : numbers) {
7         std::cout << "Number: " << num << std::endl;
8     }
9     return 0;
10 }
```

Listing 3. A simple C++ program.

For web developers, there is `jsstyle` which supports JavaScript and TypeScript natively:

```
1 interface User {
2     id: number;
3     name: string;
```

```
4 }  
5  
6 const fetchUser = async (id: number): Promise<User> => {  
7     const response = await fetch(`/api/users/${id}`);  
8     const data = await response.json();  
9     return data;  
10 };
```

Listing 4. A modern TypeScript function.

Finally, you can even showcase LaTeX code itself using the `latexstyle`:

```
1 \begin{problem}[1: Signal Analysis]  
2 Calculate the energy and power of the given signal  $x(t) = e^{-2t}u(t)$   
   $$.  
3 \end{problem}
```

Listing 5. A snippet of LaTeX code.

4 Drawing, Figures, and Tables

This section demonstrates how to use the built-in packages to create high-quality plots, draw circuits, insert images, and create beautiful tables.

4.1 Plotting Graphs with PGFPlots

The template comes pre-configured with `pgfplots` and custom styles like `snsaxis` (for continuous signals) and `snsdiscrete` (for discrete signals).

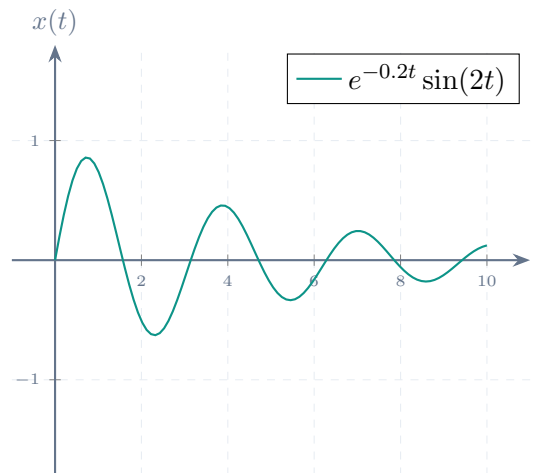


Figure 1. A continuous-time damped sine wave plotted using PGFPlots.

4.2 Drawing Circuits with CircuitikZ

You can use `circuitikz` to draw electrical circuits directly in $\text{\LaTeX}$.

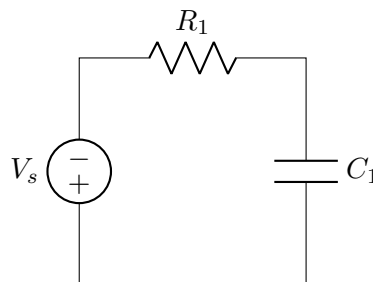


Figure 2. A simple RC circuit drawn using circuitikz.

4.3 Inserting Pre-existing Images

To include an existing image file (e.g., PNG, JPG), use the `\includegraphics` command. Make sure to place your images in the `images` directory.



Figure 3. Inserting an external image file (University Logo).

4.4 Creating Elegant Tables

The `booktabs` package is included to create beautiful, professional-looking tables without vertical lines.

Table 1. A comparison of continuous and discrete signal properties.

Property	Continuous-Time	Discrete-Time
Independent Variable	t (Real)	n (Integer)
Delta Function	Dirac Delta $\delta(t)$	Kronecker Delta $\delta[n]$
Fourier Transform	CTFT	DTFT